Section 2

Control Statements in Java

Control Statements in Java

- In Java the control statements are classified into:

Selection statements / Conditional statements	**Iteration Statements**	**Jump Statements**
if	while	break
if…else	do-while	continue
if … else if	for	return
switch		

if Statement

- if statement syntax is as follows:

```
if(condition or boolean expression)
{
   statement(s);
}
```

- If the condition is evaluated to “true”, the statements in the if block are executed.

If Statement...

Example:

```
public class Test {
  public static void main(String args[]){
    int x = 10;
    if( x < 20 ){
      System.out.println("This is if statement");
    }
  }
}
```

This would produce the following result:

```
This is if statement
```

The if...else Statement

- An if statement can be followed by an optional ***else*** statement, which executes when the boolean expression is false.
- The syntax of if...else is:

```
if(condition){
//Executes when the boolean expression is true
 }
Else {
//Executes when the boolean expression is false
 }
```

- If the condition is evaluated to “false”, the statements in the else block are executed

The if...else Statement...

Example:

```
public class Test {
   public static void main(String args[]){
      int x = 30;
      if( x < 20 ){
         System.out.print("This is if statement");
      }
   else{
         System.out.print("This is else statement");
      }
   }
}
```

This would produce the following result:

```
This is else statement
```

Nested if Statement

Syntax:

```
if(condition)
{
        statement(s);
        if(condition)
        {
                statement(s);
        }
        else
        {
                statement(s);
        }
}
else
{
        statement(s);
}
```

Outer if

Inner if

Inner else

Outer else

Nested if...

Example:

```
public class Test {
   public static void main(String args[]){
      int x = 30;
      int y = 10;
      if( x == 30 ){
         if( y == 10 ){
            System.out.print("X = 30 and Y = 10");
          }
       }
    }
}
```

This would produce the result: X = 30 and Y = 10

Dangling-*else* Problem

- The Java compiler always associates an else with the immediately preceding if unless told to do otherwise by the placement of braces ({and }).
- This behavior can lead to what is referred to as the dangling-else problem. For example,

```
if(x > 5)
    if(y > 5)
        System.out.println("x and y are > 5");
    else
        System.out.println("x is <= 5");
```

- What will be the output if x = 3 and y = 6, x = 8 and y = 4?

if…else if Statements

- The if…else if ladder is based on the nested if's concept.
- It is used to test for multiple conditions.
- ***Syntax:***

```
if(condition)
{
   statement(s);
}
else if(condition)
{
   statement(s);
}
…
else
{
   statement(s);
}
```

Example:

```
public class Test {
   public static void main(String args[]){
      int x = 30;
      if( x == 10 ){
         System.out.print("Value of X is 10");
      }
    else if( x == 20 ){
         System.out.print("Value of X is 20");
      }
```

Example (Cont'd...)

```
else if( x == 30 ){
        System.out.print("Value of X is 30");
     }
    else{
        System.out.print("This is else statement");
     }
   }
}
```

This would produce the result: Value of X is 30

Conditional Operator (? :)

- Also known as the ternary operator.
- This operator consists of three operands and is used to evaluate boolean expressions.
- The operator is written as:

  variable = (boolean expression) ? value **if true** : value **if false**

Example:

```
int h = (hour == 0 || hour == 12)? 12:hour%12;
String s = (hour < 12)? “AM”:”PM”;
```

switch Statement

- A *switch* statement allows a variable to be tested for equality against a list of values.
- Each value is called a case, and the variable being switched on is checked for each case.
- “switch” statement is a multi-way branch statement.
- It is more efficient than an if-else if ladder.

switch Statement…

- The syntax for “switch” statement is as follows:

```
switch(expression)
{
    case value1:
      statement(s);
      break;
    case value2:
      statement(s);
      break;
    …
    default:
      statement(s);
}
```

switch Statement…

Example:

```
public class Test {
   public static void main(String args[]) {
      char grade = 'C';
      switch(grade) {
         case 'A' :
            System.out.println("Excellent!");
            break;
         case 'B' :
         case 'C' :
            System.out.println("Well done");
            break;
       case 'D' :
            System.out.println("You passed");
          break;
```

Example (Cont'd…)

```
   case 'F' :
         System.out.println("Failed try again");
         break;
       default :
         System.out.println("Invalid grade");
     }
     System.out.println("Your grade is " + "\"" + grade + "\"");
   }
}
```

This would produce the following result:

```
Well done
Your grade is  'C'
```

while Loop

- "while" loop is used to loop through a set of statements until a certain condition is false.
- Syntax is as shown below:

```
while(condition or boolean expression)
{
    statement(s);
}
```

while Loop…

Example:

```
public class Test {
   public static void main(String args[]) {
      int x = 10;
      while( x < 20 ) {
         System.out.print("value of x : " + x );
         x++;
         System.out.print("\n");
      }
   }
}
```

Output:

```
value of x : 10
value of x : 11
value of x : 12
value of x : 13
value of x : 14
value of x : 15
value of x : 16
value of x : 17
value of x : 18
value of x : 19
```

do-while Loop

- do-while loop is also used to loop through a set of statements like the while loop.
- But the only difference between do-while and while loop is, all the statements in the do-while loop are executed at least once.
- Syntax is as shown below:

```
do
{
   statement(s);
}while(condition or boolean expression);
```

do-while Loop…

Example:

```
public class Test {
   public static void main(String args[]){
      int x = 10;
      do{
         System.out.print("value of x : " + x );
         x++;
         System.out.print("\n");
      }while( x < 20 );
   }
}
```

Output:

```
value of x : 10
value of x : 11
value of x : 12
value of x : 13
value of x : 14
value of x : 15
value of x : 16
value of x : 17
value of x : 18
value of x : 19
```

Sentinel-controlled while repetition

- Used when the number of repetition is not known in advance.
- Sentinel-controlled loop uses a special value called a **sentinel value**(also called a **signal value**, a **dummy value** or a **flag value**) to indicate end of data entry.

Example

write a program that reads and calculates the sum of an unspecified number of integers. Use the input 0 (sentinel value) to signify the end of the input.

Sample output:

```
Enter an integer (the input ends if it is 0): 2
Enter an integer (the input ends if it is 0): 3
Enter an integer (the input ends if it is 0): 4
Enter an integer (the input ends if it is 0): 0
The sum is 9
```

Example (Cont'd…)

```
Import java.util.Scanner;
public class SentinelValue {
  public static void main(String[] args) {
    // Create a Scanner object
    Scanner input = new Scanner(System.in);
    // Read an initial data
    System.out.print( "Enter an integer (the input
                              ends if it is 0): ");
    int data = input.nextInt();
```

Example (Cont'd...)

```
// Keep reading data until the input is 0
   int sum = 0;
   while(data !=  0) {
         sum += data;
       // Read the next data
         System.out.print( "Enter an integer (the input
                                ends if it is 0): ");
     data = input.nextInt();
  }
    System.out.println("The sum is "+ sum);
  }
}
```

Exercise

A class of unknown number of students took a quiz. The grades (integers in the range 0–100) for this quiz are available to you.

The class average is equal to the sum of the grades divided by the number of students.

The program for solving this problem on a computer must input each grade, keep track of the total of all grades input, perform the averaging calculation and print the result.

Develop a class-averaging program that processes grades for these arbitrary number of students each time it’s run.

Look at the sample output below:

Exercise(Cont'd…)

Sample output:

```
Enter grade or -1 to quit: 97
Enter grade or -1 to quit: 88
Enter grade or -1 to quit: 72
Enter grade or -1 to quit: -1
Total of the 3 grades entered is 257
Class average is 85.67
```

for Loop

- for repetition statement is used to iterate/loop through a set of statements.
- A for loop is useful when you know how many times a task is to be repeated.
- Syntax of “for” loop is as shown below:

```
for(initialization; condition; increment/decrement)
{
   statement(s);
}
```

for Loop…

Example:

```
public class Test {
   public static void main(String args[]) {
      for(int x = 10;  x < 20;  x = x + 1) {
       System.out.print("value of x : " + x );
        System.out.print("\n");
      }
   }
}
```

Output:

```
value of x : 10
value of x : 11
value of x : 12
value of x : 13
value of x : 14
value of x : 15
value of x : 16
value of x : 17
value of x : 18
value of x : 19
```

Nested Loops

- Nested loops consist of an outer loop and one or more inner loops.
- Each time the outer loop is repeated, the inner loops are reentered, and started anew.

Example:

```
public class Nested{
  Public static void main(String[] args){
     int i, j;
     for(i=0;  i<10;  i++){
         for(j=i;  j<10;  j++){
           System.out.print('*');
         }
       System.out.println();
       }
    }
}
```

Output:

```
* * * * * * * * * * *
* * * * * * * * * *
* * * * * * * * *
* * * * * * *
* * * * * *
* * * * *
* * * *
* * *
* *
*
```

Enhanced for(for each) loop in Java

- As of java5 the enhanced for loop was introduced.
- This is mainly used to iterate all the elements of arrays and collections.
- The syntax of enhanced for loop is:

```
for(declaration: expression) {
//Statements
 }
```

Enhanced for(for each) loop in Java…

Example:

```
public class Test {
   public static void main(String args[]){
      int [] numbers = {10, 20, 30, 40, 50};
      for(int x : numbers ){
         System.out.print( x );
         System.out.print(", ");
      }
```

Example (Cont'd…)

```
System.out.print("\n");
 String [] names ={"James", "Larry", "Tom", "Moha"};
    for( String name : names ) {
      System.out.print( name );
      System.out.print(", ");
    }
  }
}
```

Output:
10, 20, 30, 40, 50,
James, Larry, Tom, Moha,

break statement

- In java break keyword can be used to exit from the case of a switch statement and from a loop.
- “if” statements are used for checking a condition to break from the loop using the keyword “break”.

Example1:

```
for(int x = 0; x < 10; x++)
{
   if(x == 5)
   break;
   System.out.println(x);
}
```

continue statement

- The ***continue*** keyword can be used in any of the loop control structures.
- It causes the loop to immediately jump to the next iteration of the loop.
- In a for loop, the continue keyword causes flow of control to immediately jump to the increment statement.
- In a while loop or do while loop, flow of control immediately jumps to the boolean expression.
- Similarly as in the case of “break” statement, we will check a condition using “if” statement to use the “continue” statement.

continue statement…

Example1:

```
for(int i=0;i<5;i++)
{
   if(i==3)
   continue;
   System.out.println(i);
}
```

The output will be:

0
1
2
4

Example2:

```
int [] numbers = {10, 20, 30, 40, 50};
    for(int x : numbers ) {
      if( x == 30 ) {
       continue;
       }
       System.out.print( x );
       System.out.print("\n");
    }
}
```

Output:

10
20
40
50

return statement

- The “return” keyword is used inside a method, to return a value to the calling method.
- When a return statement is executed, it returns the value to the calling method and exits the called method.
- So, the execution continues with the next line after the calling method.
- It can also be used to indicate the end of void method.

```
return;
```